



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

Facultad de Ciencias

Compiladores

Árboles de sintaxis abstracta

Práctica 4

Presenta:

Ramírez Juárez María Fernanda - 321204747

Rojo Peña Manuel Ianluck - 118005762

Profesor:

Pablo Martínez Yessica Janeth

Ayudante de laboratorio:

Salazar González Pedro Yamil

Grupo: 7010, 2027-1

Fecha de entrega:

2 de octubre, 2026

Objetivo

En esta práctica extendimos el analizador sintáctico descendente recursivo de la Práctica 3 para que, además de validar la sintaxis de un programa en MINIC, construya un Árbol de Sintaxis Abstracta (AST) que represente su estructura esencial. La Práctica 3 respondía a la pregunta *¿el programa está bien escrito?*; ahora el parser también responde *¿cómo está construido?*.

El AST es la representación que consumirán el análisis semántico y la generación de código, por lo que en esta entrega no hacemos validaciones semánticas, tabla de símbolos, verificación de tipos ni generación de código. Concretamente, el objetivo es:

1. Diseñar una jerarquía de nodos para programas, declaraciones, sentencias y expresiones
2. Modificar las funciones del parser para que devuelvan el nodo que reconocen
3. Reflejar la precedencia y la asociatividad de los operadores en la forma del árbol
4. Conservar la línea y la columna de cada construcción
5. Definir un único propietario para cada recurso dinámico y liberar el árbol completo
6. Imprimir el árbol en un formato jerárquico y determinista
7. No entregar jamás un árbol parcial cuando existan errores léxicos o sintácticos

Evolución respecto de la Práctica 3

Módulos modificados o agregados

- `ast.h` y `ast.c`: módulo nuevo con los tipos de nodo, la lista dinámica de sentencias, los constructores, la impresión del árbol y su liberación recursiva.
- `grammar.c` y `grammar.h`: todas las funciones de la gramática pasaron de `void` a `ASTNode *`, y construyen y devuelven el nodo que reconocen. `parse_program` ahora devuelve la raíz.
- `parser.h` y `parser.c`: `parser_parse_program` recibe ahora un parámetro de salida con la raíz del árbol. El resto de `parser.c` (flujo de tokens, diagnósticos y sincronización) no cambió.
- `main.c`: imprime `AST:`, el árbol y el mensaje final solo cuando el análisis fue correcto, y libera el árbol.
- `lexer`, `token`, `keywords`, `buffer_manage` y `parser_internal.h` se conservan sin cambios.

Cambio de interfaz en parser_parse_program

```

1  /*
2  * - PARSE_OK: *out_root es la raíz del AST; el llamador es su
3  *   propietario y debe liberarlo con ast_destroy.
4  * - PARSE_INVALID / PARSE_INTERNAL_FAILURE: *out_root es NULL; el
5  *   parser ya libero cualquier arbol parcial.
6  */
7  ParseResult parser_parse_program(Parser *parser, ASTNode **out_root);

```

Código 1: Interfaz modificada de parser.h.

Mantuvimos el contrato de errores de la Práctica 3 (ParseResult) y entregamos el árbol por un parámetro de salida. De este modo es imposible que un llamador reciba un AST junto con un resultado inválido: si el resultado no es PARSE_OK, *out_root vale NULL.

Función parse_statement_list

La función parse_statement_list de la Práctica 3 desapareció. Su ciclo pasó a parse_program, porque ahora cada sentencia reconocida debe agregarse a una lista de nodos y, al final, parse_program decide si entrega la raíz o NULL. Mantener una función aparte solo habría obligado a pasarle la lista y un indicador de fallo.

Diseño del AST

Tipos de nodo, operadores y tipos declarados

```

1  typedef enum {
2      AST_PROGRAM, AST_BLOCK,
3      AST_VARIABLE_DECLARATION, AST_ASSIGNMENT, AST_PRINT,
4      AST_IF, AST_WHILE,
5      AST_BINARY_EXPRESSION, AST_UNARY_EXPRESSION,
6      AST_IDENTIFIER, AST_INTEGER_LITERAL, AST_BOOLEAN_LITERAL
7  } ASTNodeType;
8
9  typedef enum { AST_TYPE_INT, AST_TYPE_BOOL } ASTDeclaredType;
10
11 typedef enum {
12     OP_ADD, OP_SUBTRACT, OP_MULTIPLY, OP_DIVIDE,
13     OP_LESS, OP_LESS_EQUAL, OP_GREATER, OP_GREATER_EQUAL,
14     OP_EQUAL, OP_NOT_EQUAL, OP_AND, OP_OR
15 } BinaryOperator;
16
17 typedef enum { OP_NEGATE } UnaryOperator;

```

Código 2: Enumeraciones de ast.h.

Usamos un único tipo de nodo para todas las expresiones binarias, y una enumeración BinaryOperator que clasifica el operador, de modo que el AST nunca depende del lexema. El tipo declarado de una variable se representa con ASTDeclaredType en lugar de reutilizar TokenType, así el módulo del AST no depende del lexer.

Estructura general del nodo

Cada nodo guarda su tipo, su posición y una unión con los datos específicos de su categoría:

```

1 struct ASTNode {
2     ASTNodeType type;
3     int line;
4     int column;
5
6     union {
7         struct { ASTNodeList statements; } program;
8         struct { ASTNodeList statements; } block;
9         struct { ASTDeclaredType declared_type; char *name;
10                ASTNode *initializer; } variable_declaration;
11        struct { char *name; ASTNode *value; } assignment;
12        struct { ASTNode *expression; } print_statement;
13        struct { ASTNode *condition, *then_branch,
14                *else_branch; } if_statement;
15        struct { ASTNode *condition, *body; } while_statement;
16        struct { BinaryOperator operator;
17                ASTNode *left, *right; } binary;
18        struct { UnaryOperator operator; ASTNode *operand; } unary;
19        struct { char *name; } identifier;
20        struct { char *lexeme; } integer_literal;
21        struct { int value; } boolean_literal;
22    } data;
23 };

```

Código 3: Estructura ASTNode (resumida).

La ausencia de inicializador en una declaración y de rama `else` se representan con `NULL`. El literal entero conserva su lexema como cadena, de modo que no se pierde información ni se impone un tamaño de entero antes del análisis semántico.

Información de posición

Todos los nodos almacenan línea (desde 1) y columna (desde 0), con la misma convención del lexer. La posición de cada nodo es la que fija el enunciado:

- Program: la de la primera sentencia o, si el programa está vacío, la de `TOKEN_EOF`.
- Block, if, while y print: la de `{`, `if`, `while` y `print`, respectivamente.
- Declaración: la del identificador declarado (será la usada para reportar redeclaraciones).
- Asignación: la del identificador destino.
- Expresión binaria: la del token del operador. Expresión unaria: la del `-`.
- Identificadores y literales: la de su propio token.

Los paréntesis no generan nodo, así que la expresión interna conserva su posición original. Como el lexer usa `size_t` para línea y columna y el AST usa `int`, la conversión se hace en un solo lugar de `grammar.c` (`current_line` y `current_column`). Las posiciones no se imprimen, pero quedan almacenadas para los diagnósticos de etapas posteriores.

Lista dinámica de sentencias

Program y Block contienen un `ASTNodeList`, un arreglo dinámico que duplica su capacidad al llenarse (capacidad inicial de 8), por lo que no existe un límite fijo de sentencias.

```
1 int
2 ast_node_list_append(ASTNodeList *list, ASTNode *node)
3 {
4     if (list == NULL || node == NULL)
5         return 0;
6
7     if (list->count == list->capacity) {
8         size_t new_capacity = list->capacity ? list->capacity * 2 : 8;
9         ASTNode **items;
10
11         if (new_capacity > SIZE_MAX / sizeof(*items))
12             return 0;
13         items = realloc(list->items, new_capacity * sizeof(*items));
14         if (items == NULL)
15             return 0;
16         list->items = items;
17         list->capacity = new_capacity;
18     }
19
20     list->items[list->count++] = node;
21     return 1;
22 }
```

Código 4: `ast_node_list_append`.

El resultado de `realloc` se guarda primero en una variable temporal, de modo que si falla el arreglo anterior no se pierde. Si la operación falla, el nodo sigue perteneciendo a quien llamó.

Propiedad de la memoria

Definimos un único propietario para cada recurso dinámico:

- El AST guarda copias propias de nombres de identificadores y de lexemas enteros; nunca apuntadores a lexemas de tokens. Cada constructor que recibe una cadena la copia.
- Cada nodo es propietario de sus hijos, `Program` y `Block` lo son de sus listas, y la raíz lo es de todo el árbol.
- **Constructores:** si devuelven un nodo válido, este adquiere la propiedad de los hijos (o de la lista). Si devuelven `NULL`, los hijos siguen siendo de quien llamó, que debe liberarlos.
- `ast_node_list_append`: si termina bien, la lista adquiere el nodo; si falla, el nodo sigue siendo del llamador.
- `ast_destroy`: acepta `NULL` y libera recursivamente cadenas, hijos, listas y el propio nodo. Liberar la raíz libera todo el árbol.

La propiedad de los tokens sigue el contrato de la Práctica 3. El punto delicado es que `parser_advance` destruye el token `previous` anterior, así que cualquier cadena que deba sobrevivir debe copiarse *antes* de avanzar. Por eso las funciones que construyen nodos con nombre (`parse_declaration` y `parse_assignment`) copian el lexema del identificador antes de llamar a `parser_advance`, y los literales e identificadores de `parse_primary` se crean antes de consumir su token.

```

1  line = current_line(parser);
2  column = current_column(parser);
3  name = copy_string(parser->current.lexeme);
4  if (name == NULL)
5      return fail_memory(parser);
6  parser_advance(parser); /* el token actual ya no se necesita */

```

Código 5: Copia del nombre antes de avanzar en `parse_declaration`.

Construcción del AST desde el parser

Convención de errores

Cada función de `grammar.c` devuelve el nodo que reconoce o NULL. Un NULL siempre viene acompañado de un error ya reportado (`had_error`) o de una falla interna (`internal_failure`). Quien recibe NULL libera los nodos parciales que todavía posee y propaga el NULL hacia arriba.

Cuando un constructor `ast_create_*` devuelve NULL, `fail_memory` imprime el mensaje de memoria, activa `internal_failure` y detiene el análisis, de modo que el resultado final es `PARSE_INTERNAL_FAILURE` (código de salida 2).

Sentencias

- `parse_statement`: elige la producción por el token de anticipación, igual que en la Práctica 3, devuelve el nodo y sincroniza si quedó activo `panic_mode`.
- `parse_declaration`: devuelve un nodo con el tipo, el nombre y el inicializador opcional (sin crear un nodo de asignación aparte).
- `parse_assignment`, `parse_print_statement`, `parse_if_statement` y `parse_while_statement`: construyen su nodo después de reconocer todos sus componentes. Si alguno falla, liberan los que ya construyeron.
- `parse_block`: sigue analizando sentencias tras un error para recuperarse, pero descarta la lista y devuelve NULL si cualquiera falló.
- `parse_program`: acumula las sentencias en una lista, y devuelve la raíz solo si no hubo ningún error léxico, sintáctico o interno.

```

1  if (failed || parser->had_error || parser->internal_failure) {
2      ast_node_list_destroy(&list);
3      return NULL;
4  }

```

Código 6: Resultado de `parse_program` ante errores.

Un error léxico no interrumpe el análisis (el token `ERROR` se reporta y se descarta), pero activa `had_error`, por lo que `parse_program` nunca entrega árbol aunque las sentencias restantes sean válidas.

Expresiones binarias

En la Práctica 3 había seis funciones casi idénticas, una por nivel de precedencia. Ahora todas comparten una función genérica que recibe la función del nivel inmediato superior y una tabla de pares (token, operador):

```

1 static ASTNode *
2 parse_binary_level(Parser *parser, ParseFn next,
3                   const BinaryOpEntry *ops, size_t count)
4 {
5     ASTNode *left = next(parser);
6
7     while (left != NULL && !parser->panic_mode
8           && !parser->internal_failure) {
9         const BinaryOpEntry *found = NULL;
10        ASTNode *right, *node;
11        size_t i;
12        int line, column;
13
14        for (i = 0; i < count; i++)
15            if (parser_check(parser, ops[i].token)) {
16                found = &ops[i];
17                break;
18            }
19        if (found == NULL)
20            break;
21
22        line = current_line(parser);
23        column = current_column(parser);
24        parser_advance(parser);
25
26        right = next(parser);
27        if (right == NULL) {
28            ast_destroy(left);
29            return NULL;
30        }
31
32        node = ast_create_binary(found->op, left, right, line, column);
33        if (node == NULL) {
34            ast_destroy(left);
35            ast_destroy(right);
36            return fail_memory(parser);
37        }
38        left = node;
39    }
40    return left;
41 }

```

Código 7: `parse_binary_level`, núcleo de las expresiones binarias.

Este ciclo construye el árbol de izquierda a derecha: el nodo recién creado pasa a ser el operando

izquierdo de la siguiente iteración. Por eso $10 - 5 - 2$ produce `Subtract(Subtract(10, 5), 2)` y no `Subtract(10, Subtract(5, 2))`. La posición del nodo es la del operador, que se guarda *antes* de avanzar. Los niveles son `parse_or_expression`, `parse_and_expression`, `parse_equality_expression`, `parse_comparison_expression`, `parse_additive_expression` y `parse_multiplicative_expression`, cada uno con su tabla. La precedencia sigue saliendo del orden en que se llaman, como en la Práctica 3.

Expresiones unarias y primarias

- `parse_unary_expression`: si hay `-`, guarda su posición, avanza, se llama recursivamente y crea un nodo `Negate`. La recursión hacia la derecha conserva la asociatividad derecha de la gramática, de modo que `-x` produce `Negate(Negate(x))`.
- `parse_primary`: crea un nodo de entero, booleano o identificador. Para una expresión entre paréntesis devuelve directamente el nodo de la expresión interna, sin nodo de paréntesis, pues su efecto queda reflejado en la estructura. Así `(5 + 2) * 3` produce `Multiply(Add(5, 2), 3)`.

Manejo de errores y recuperación

La recuperación de la Práctica 3 se conserva sin cambios: `parser_error` imprime un único diagnóstico por sentencia gracias a `panic_mode`, y `parser_sync` descarta tokens hasta un punto seguro. Las reglas adicionales de esta práctica son:

- Si hubo al menos un error, el programa se marca como inválido y no se entrega árbol.
- Los nodos parciales se liberan en cuanto dejan de poder formar un árbol válido, en la función que los posee.
- Los diagnósticos van a `stderr`, `stdout` permanece vacío y el código de salida es distinto de cero.
- Por ejemplo, ante `x = 5 + ;` no se construye ningún nodo `Add` con operando derecho inexistente: `parse_primary` devuelve `NULL`, `parse_binary_level` libera el operando izquierdo y propaga `NULL`, y `parse_assignment` libera su nombre copiado.

Un detalle importante es el de `parse_if_statement`: tras un error en la rama verdadera, el parser igualmente comprueba si sigue un `else` para no dejarlo huérfano y reportar un error de más. Después libera condición y ramas y devuelve `NULL`.

Impresión y liberación del AST

`ast_print` imprime el árbol con el formato canónico: dos espacios por nivel, un elemento por línea, sin conectores gráficos, tabuladores, líneas vacías ni direcciones. El encabezado `AST:` y el mensaje final son responsabilidad de `main.c`. Las etiquetas `Condition`, `Then`, `Else` y `Body` solo se imprimen para identificar la función de cada hijo y no son nodos reales del AST.

Para el programa

```
int x = 5;
x = x + 3;
print(x);
```

el programa imprime:

AST:

Program

VariableDeclaration(int, x)

Integer(5)

Assignment(x)

Add

Identifier(x)

Integer(3)

Print

Identifier(x)

Programa sintacticamente correcto.

`ast_destroy` recorre el árbol según el tipo de cada nodo: libera las cadenas copiadas, destruye recursivamente los hijos y las listas de `Program` y `Block`, y finalmente el propio nodo. Aceptar `NULL` simplifica los constructores y la limpieza de errores, porque una declaración sin inicializador o un `if` sin `else` se liberan sin casos especiales.

Flujo de `main.c`. En modo normal, si `parser_parse_program` devuelve `PARSE_OK` se imprimen AST:, el árbol y el mensaje de éxito. En cualquier caso se llama a `ast_destroy`, que acepta la raíz `NULL`, y después se liberan el parser y el lexer.

Resultados y pruebas

Cambios en el Makefile

Respecto de la Práctica 4, el `Makefile` cambió en los siguientes puntos:

- Se agregaron las variables `UNIT7_AST_INPUT_DIR` y `UNIT_AST_EXPECTED_DIR`, que indican, respectivamente, la ubicación de los programas MiniC utilizados como casos de prueba del AST y la ubicación de los archivos `.out` que contienen las salidas esperadas.

```
UNIT_AST_INPUT_DIR = tests/unit_tests/ast/inputs
```

```
UNIT_AST_EXPECTED_DIR = tests/unit_tests/ast/expected
```

- Se modificó el objetivo `test-unit` para incluir `test-unit-ast`, de modo que las pruebas del AST se ejecuten junto con las pruebas unitarias del Lexer y del Parser.

```
test-unit: test-unit-lexer test-unit-parser test-unit-ast ...
```

- Se agregó el objetivo `test-unit-ast`, el cual utiliza la plantilla `run_test_dir` para ejecutar los casos de prueba ubicados en `tests/unit_tests/ast/inputs` y comparar sus resultados con los archivos correspondientes de `tests/unit_tests/ast/expected`.

```
test-unit-ast: $(TARGET)
$(call run_test_dir,$(UNIT_AST_INPUT_DIR),$(UNIT_AST_EXPECTED_DIR),,unit-ast)
```

- Las pruebas propias del AST pueden ejecutarse de manera independiente mediante `make test-unit-ast`. Al estar incluidas dentro de `test-unit`, también se ejecutan mediante `make test-unit` y, de forma indirecta, mediante `make test`, que agrupa las pruebas de la interfaz de línea de comandos, las pruebas unitarias y las pruebas de argumentos.
- Se incorporó la prueba pública correspondiente a la Práctica 4 mediante la variable `PUBLIC_P4_SCRIPT`, que indica la ubicación del script encargado de ejecutar los casos públicos.

```
PUBLIC_P4_SCRIPT = tests/public/p4/run_public_tests.sh
```

- Se agregó el objetivo `test-p4`, encargado de ejecutar el script de pruebas públicas, almacenar su salida en `build/public-p4.log` y comprobar mediante `grep` que no existan pruebas fallidas.

```
test-p4: $(TARGET)
@mkdir -p $(BUILD_DIR)
@echo "==== public-p4 ====="
@sh $(PUBLIC_P4_SCRIPT) ./${TARGET} | tee $(BUILD_DIR)/public-p4.log
@grep -q '^Fallidas: 0$$' $(BUILD_DIR)/public-p4.log
```

Pruebas públicas de la Práctica 4

Al ejecutar las pruebas proporcionadas en `make test-p4` los resultados todos pasaron sin problemas.

```
make test-p4
==== public-p4 =====
[OK] p01_empty
[OK] p02_declarations
[OK] p03_initializers
[OK] p04_assignment_print
[OK] p05_precedence
[OK] p06_unary_parentheses
[OK] p07_if_else
[OK] p08_while_block
[OK] p09_nested_blocks
[OK] p10_complete_program
[OK] p11_syntax_error
[OK] p12_lexical_error
```

Aprobadas: 12

Fallidas: 0

Las pruebas públicas comparan `stdout` contra el AST canónico, de modo que verifican la estructura del árbol y no solo que el programa sea aceptado.

Pruebas propias del AST

Además de las pruebas públicas proporcionadas para la práctica, se diseñaron pruebas propias enfocadas específicamente en verificar la correcta construcción del Árbol de Sintaxis Abstracta (AST).

Estas pruebas buscan comprobar aspectos que pueden presentar errores durante la implementación del parser, principalmente la precedencia y asociatividad de operadores, el manejo de operadores unarios, los paréntesis y la asociación de sentencias condicionales y ciclos.

Las pruebas se organizaron de la siguiente manera:

```
tests/unit_tests/ast/
inputs/
  ut_ast_assoc_add.mc
  ut_ast_assoc_mul.mc
  ut_ast_mixed_operators.mc
  ut_ast_multiple_unary.mc
  ut_ast_nested_parentheses.mc
  ut_ast_boolean_expression.mc
  ut_ast_dangling_else.mc
  ut_ast_single_statement_while.mc
  ut_ast_deep_expression.mc
expected/
  ut_ast_assoc_add.out
  ut_ast_assoc_mul.out
  ut_ast_mixed_operators.out
  ut_ast_multiple_unary.out
  ut_ast_nested_parentheses.out
  ut_ast_boolean_expression.out
  ut_ast_dangling_else.out
  ut_ast_single_statement_while.out
  ut_ast_deep_expression.out
```

Cada archivo de entrada contiene un programa MiniC y cada archivo .out contiene la representación esperada del AST generado por el compilador.

Asociatividad de operadores

La primera prueba utiliza la expresión:

```
int x = 10 + 5 + 2;
```

El resultado esperado es:

```
Add
Add
Integer(10)
Integer(5)
Integer(2)
```

Esta prueba permite verificar que las operaciones de suma se construyen de forma asociativa hacia la izquierda, es decir, como $(10 + 5) + 2$.

De manera similar, se probó la multiplicación mediante:

```
int x = 2 * 3 * 4;
```

para comprobar que el árbol se construye como $(2 * 3) * 4$.

Precedencia de operadores

Para comprobar que los operadores con mayor precedencia se procesan primero, se utilizó:

```
int x = 2 + 3 * 4 - 5;
```

El árbol esperado es:

```
Subtract
Add
Integer(2)
Multiply
Integer(3)
Integer(4)
Integer(5)
```

Esto representa la expresión $((2 + (3 * 4)) - 5)$, demostrando que la multiplicación tiene mayor precedencia que la suma y la resta.

Operadores unarios

También se verificó el manejo de operadores unarios mediante:

```
int x = ---5;
```

El AST esperado contiene tres nodos **Negate** anidados:

```
Negate
Negate
Negate
Integer(5)
```

Esta prueba permite comprobar que el parser puede manejar operadores unarios consecutivos y construir correctamente su estructura recursiva.

Paréntesis anidados

Para comprobar que los paréntesis modifican correctamente la estructura de la expresión, se utilizó:

```
int x = (((2 + 3) * 4) - 5);
```

El AST generado debe conservar la estructura indicada por los paréntesis:

```
Subtract
Multiply
Add
Integer(2)
Integer(3)
Integer(4)
Integer(5)
```

Precedencia de operadores booleanos

Se utilizó la expresión:

```
bool resultado = 10 > 5 && 3 < 8 || false;
```

El árbol esperado es:

```
Or
And
Greater
Integer(10)
Integer(5)
Less
Integer(3)
Integer(8)
Boolean(false)
```

Con esta prueba se verifica que las operaciones relacionales se encuentran dentro de la operación `And`, y que `And` tiene mayor precedencia que `Or`.

Asociación del `else`

Para comprobar el comportamiento del `else` en condiciones anidadas se utilizó:

```
if (x > 0)
if (x > 5)
print(1);
else
print(2);
```

El AST esperado asocia el `else` con el `if` más cercano. Esto permite verificar que el parser implementa correctamente la regla de asociación de la sentencia `else`.

Ciclo while con una sola sentencia

También se probó un `while` cuyo cuerpo no está encerrado en un bloque:

```
while (x > 0)
x = x - 1;
```

El objetivo es verificar que el parser pueda utilizar una sentencia individual como cuerpo del ciclo y que dicha sentencia aparezca correctamente dentro del nodo `Body` del AST.

Expresión compleja

Finalmente, se utilizó una expresión que combina operadores unarios, paréntesis, operadores aritméticos y división:

```
int x = -((2 + 3) * (4 - 1)) + 10 / 2;
```

Esta prueba permite comprobar simultáneamente diferentes niveles de precedencia y la construcción recursiva del AST.

Ejecución de las pruebas

Además de las pruebas públicas proporcionadas para la práctica, se diseñaron pruebas propias enfocadas específicamente en verificar la correcta construcción del Árbol de Sintaxis Abstracta (AST).

Estas pruebas buscan comprobar aspectos que pueden presentar errores durante la implementación del parser, principalmente la precedencia y asociatividad de operadores, el manejo de operadores unarios, los paréntesis y la asociación de sentencias condicionales y ciclos.

Las pruebas se organizaron de la siguiente manera:

```
tests/unit_tests/ast/
inputs/
  ut_ast_assoc_add.mc
  ut_ast_assoc_mul.mc
  ut_ast_mixed_operators.mc
  ut_ast_multiple_unary.mc
  ut_ast_nested_parentheses.mc
  ut_ast_boolean_expression.mc
  ut_ast_dangling_else.mc
  ut_ast_single_statement_while.mc
  ut_ast_deep_expression.mc
expected/
  ut_ast_assoc_add.out
  ut_ast_assoc_mul.out
  ut_ast_mixed_operators.out
  ut_ast_multiple_unary.out
  ut_ast_nested_parentheses.out
  ut_ast_boolean_expression.out
```

```
ut_ast_dangling_else.out
ut_ast_single_statement_while.out
ut_ast_deep_expression.out
```

Cada archivo de entrada contiene un programa MiniC y cada archivo `.out` contiene la representación esperada del AST generado por el compilador.

Asociatividad de operadores

La primera prueba utiliza la expresión:

```
int x = 10 + 5 + 2;
```

El resultado esperado es:

```
Add
Add
Integer(10)
Integer(5)
Integer(2)
```

Esta prueba permite verificar que las operaciones de suma se construyen de forma asociativa hacia la izquierda, es decir, como $(10 + 5) + 2$.

De manera similar, se probó la multiplicación mediante:

```
int x = 2 * 3 * 4;
```

para comprobar que el árbol se construye como $(2 * 3) * 4$.

Precedencia de operadores

Para comprobar que los operadores con mayor precedencia se procesan primero, se utilizó:

```
int x = 2 + 3 * 4 - 5;
```

El árbol esperado es:

```
Subtract
Add
Integer(2)
Multiply
Integer(3)
Integer(4)
Integer(5)
```

Esto representa la expresión $((2 + (3 * 4)) - 5)$, demostrando que la multiplicación tiene mayor precedencia que la suma y la resta.

Operadores unarios

También se verificó el manejo de operadores unarios mediante:

```
int x = ---5;
```

El AST esperado contiene tres nodos **Negate** anidados:

```
Negate  
Negate  
Negate  
Integer(5)
```

Esta prueba permite comprobar que el parser puede manejar operadores unarios consecutivos y construir correctamente su estructura recursiva.

Paréntesis anidados

Para comprobar que los paréntesis modifican correctamente la estructura de la expresión, se utilizó:

```
int x = (((2 + 3) * 4) - 5);
```

El AST generado debe conservar la estructura indicada por los paréntesis:

```
Subtract  
Multiply  
Add  
Integer(2)  
Integer(3)  
Integer(4)  
Integer(5)
```

Precedencia de operadores booleanos

Se utilizó la expresión:

```
bool resultado = 10 > 5 && 3 < 8 || false;
```

El árbol esperado es:

```
Or  
And  
Greater  
Integer(10)  
Integer(5)  
Less  
Integer(3)  
Integer(8)  
Boolean(false)
```

Con esta prueba se verifica que las operaciones relacionales se encuentran dentro de la operación **And**, y que **And** tiene mayor precedencia que **Or**.

Asociación del else

Para comprobar el comportamiento del **else** en condiciones anidadas se utilizó:

```
if (x > 0)
if (x > 5)
print(1);
else
print(2);
```

El AST esperado asocia el **else** con el **if** más cercano. Esto permite verificar que el parser implementa correctamente la regla de asociación de la sentencia **else**.

Ciclo while con una sola sentencia

También se probó un **while** cuyo cuerpo no está encerrado en un bloque:

```
while (x > 0)
x = x - 1;
```

El objetivo es verificar que el parser pueda utilizar una sentencia individual como cuerpo del ciclo y que dicha sentencia aparezca correctamente dentro del nodo **Body** del AST.

Expresión compleja

Finalmente, se utilizó una expresión que combina operadores unarios, paréntesis, operadores aritméticos y división:

```
int x = -((2 + 3) * (4 - 1)) + 10 / 2;
```

Esta prueba permite comprobar simultáneamente diferentes niveles de precedencia y la construcción recursiva del AST.

Ejecución de las pruebas

El sistema ejecuta cada archivo **.mc** y compara automáticamente la salida obtenida con el archivo **.out** correspondiente. Una prueba se considera correcta cuando ambas salidas coinciden.

De esta manera, las pruebas propias permiten validar no solamente que los programas sean sintácticamente aceptados, sino que la **estructura del AST generado sea la esperada**, especialmente en casos donde la precedencia, asociatividad o anidamiento de las expresiones puede provocar errores.

```
make test-unit-ast
==== unit-ast ====
PASS: ut_ast_assoc_add
PASS: ut_ast_assoc_mul
PASS: ut_ast_boolean_expression
PASS: ut_ast_dangling_else
PASS: ut_ast_deep_expression
```

```
PASS: ut_ast_mixed_operators
PASS: ut_ast_multiple_unary
PASS: ut_ast_nested_parentheses
PASS: ut_ast_single_statement_while
----
Total: 9 Pass: 9 Fail: 0
```

Problemas encontrados

Copia de lexemas antes de avanzar

Como `parser_advance` destruye el token anterior, el nombre de una declaración o asignación debía copiarse antes de avanzar, y el inicializador se analiza después de haber consumido el identificador. De lo contrario, el nodo habría guardado un apuntador a memoria ya liberada.

Límite por anidamiento profundo

Como en la Práctica 3, el descenso recursivo consume pila en proporción al anidamiento, y ahora también lo hacen `ast_print` y `ast_destroy`, que son recursivas sobre el árbol. Una entrada con una cantidad extrema de paréntesis o bloques anidados puede agotar la pila. No agregamos un límite de profundidad y lo dejamos registrado como limitación conocida.

Conclusiones

Esta práctica nos mostró que construir un AST sobre un parser descendente recursivo es, en gran parte, un cambio de lo que devuelve cada función. Como en la Práctica 3 ya teníamos una función por cada no terminal, los cambios se concentraron en `grammar.c` y en el nuevo módulo `ast`, mientras que el manejo de tokens de `parser.c` y el lexer prácticamente no se tocaron. Eso confirmó la decisión que tomamos al separar el flujo de tokens de la gramática.

La precedencia y la asociatividad, que antes solo se observaban de forma indirecta, ahora quedan visibles en la forma del árbol. La precedencia sigue saliendo del orden en que se llaman los niveles de expresión, y la asociatividad izquierda del ciclo que usa el nodo recién creado como operando izquierdo de la siguiente iteración. Además, al reunir los seis niveles binarios en una sola función genérica con una tabla de operadores, eliminamos código repetido y redujimos la posibilidad de errores entre niveles.

La parte más delicada fue la propiedad de la memoria. Definir quién es dueño de cada nodo, de cada lista y de cada cadena nos obligó a pensar qué pasa en cada ruta de error: si un constructor falla, los hijos siguen siendo de quien llamó, y si una función recibe NULL debe liberar lo que ya construyó antes de propagarlo. También aprendimos que el token `previous` se destruye al avanzar, por lo que los nombres deben copiarse al AST antes de consumir el token. La convención de devolver NULL ante cualquier error y no entregar nunca un árbol parcial simplificó mucho el razonamiento sobre la recuperación, porque el resultado del análisis es o un árbol completo o ninguno.

Por último, las pruebas nos recordaron que aceptar un programa no basta: hay que verificar la estructura que se construyó, como lo muestran los fallos que encontramos al integrar `main.c` y al ejecutar por error las pruebas de la práctica anterior. Como limitación, el diseño sigue siendo recursivo, así que el análisis, la impresión y la liberación consumen pila en proporción a la profundidad del anidamiento. El AST quedó con una representación correcta, modular e independiente de la impresión, lista para que el análisis semántico de la siguiente práctica la recorra y la enriquezca con tipos y símbolos.

División del trabajo

Para esta práctica, el trabajo se repartió de la siguiente manera:

El compañero *Manuel Ianluck* se encargó de modificar el `Makefile`, de crear las pruebas propias y de participar en la discusión y el diseño de la implementación del AST, en particular la definición de los tipos de nodo y del contrato de propiedad de la memoria.

Por su parte, la compañera *María Fernanda* se encargó de la implementación concreta del AST y de la integración con el parser: **[CONFIRMAR: por ejemplo, el módulo ast (constructores, listas, impresión y liberación), la modificación de las funciones de `grammar.c` para que devuelvan nodos, y los cambios en `parser.c` y `main.c`].**

La redacción del reporte, la revisión del código y las pruebas con `valgrind` las realizamos entre ambos.

Uso de herramientas de Inteligencia Artificial

Para la realización de esta práctica se utilizó la herramienta de Inteligencia Artificial ChatGPT como apoyo en la redacción y revisión del reporte. Su uso se enfocó principalmente en mejorar la claridad de algunas explicaciones, corregir errores de redacción y reorganizar determinadas partes del texto para facilitar su comprensión.

El contenido técnico del reporte, así como la implementación y las pruebas realizadas durante la práctica, fueron desarrollados a partir del trabajo realizado en el proyecto. La herramienta se utilizó únicamente como apoyo para presentar dicha información de una manera más clara y estructurada.

Se utilizó el siguiente prompt:

```
Ayúdame a mejorar la redacción de este pdf.  
Mantén la información técnica y el significado original, pero  
haz que el texto sea más claro, formal y fácil de entender.  
No agregues información que no esté presente en el texto original.
```

References

- [1] `cppreference.com`. (n.d.). C language. Disponible en <https://en.cppreference.com/c/language>
- [2] `cppreference.com`. (n.d.). C11. Disponible en <https://en.cppreference.com/c/11>

- [3] Free Software Foundation. (n.d.). GNU make. Disponible en <https://www.gnu.org/software/make/manual/make.html>